

NOMBRE DEL LABORATORIO: FILE PATH TRAVERSAL, VALIDATION OF FILE EXTENSION WITH NULL BYTE BYPASS.

Programación Wap

Prof: Servando Lopez Contreras

David Campos Morado – 183511
Ingeniería en Tecnologías de la Información

Ficha Técnica del Laboratorio

- **Nombre del laboratorio:** File path traversal, validation of file extension with null byte bypass.
- **Nivel:** Practitioner (Intermedio).
- **Tipo de explotación:** File Path Traversal (Evasión de validación de extensión mediante Byte Nulo).
- **Objetivo del laboratorio:** Evadir el filtro de seguridad que exige una extensión de archivo específica, utilizando un byte nulo (%00) para truncar la ruta, lograr leer el archivo /etc/passwd y conseguir el "Lab Solved".

Marco Teórico y Fundamentación Técnica

Siguiendo la línea del OWASP Top 10, esta vulnerabilidad es un fallo crítico de *Broken Access Control*. Permite manipular la ruta de los archivos para salir de las carpetas públicas del servidor web.

En este caso, la defensa de los desarrolladores fue asegurarse de que cualquier archivo solicitado termine exactamente con la extensión correcta (por ejemplo, .jpg o .png). El problema es que esta validación es superficial y no tiene en cuenta cómo funcionan las APIs del sistema operativo subyacente (backend) al procesar cadenas de texto.

En lenguajes basados en C o en APIs de sistemas operativos a bajo nivel, el **Byte Nulo** (%00 en formato URL) es un indicador de terminación de cadena (null-terminator).

Este error compromete directamente la **Tríada CIA**:

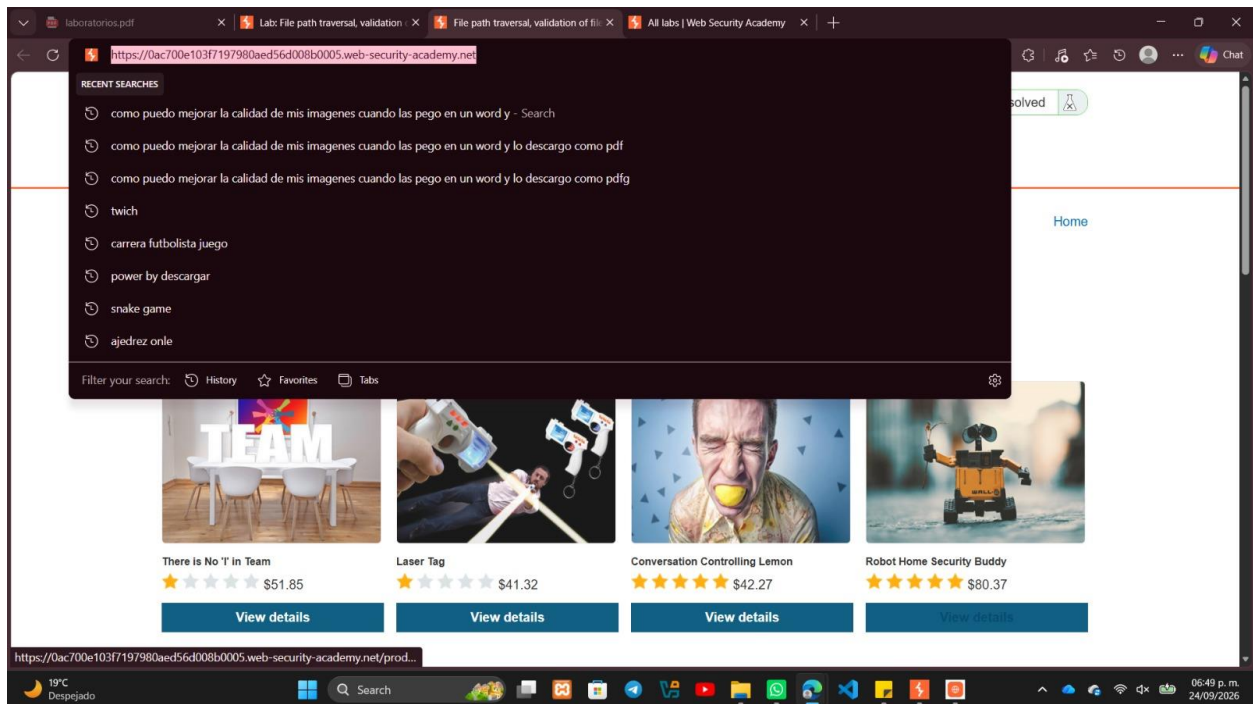
- **Confidencialidad (Crítico):** Nos permite engañar al sistema para leer archivos restringidos, exponiendo información confidencial.
- **Integridad (Bajo):** El ataque es de lectura, por lo que no altera los archivos del servidor.
- **Disponibilidad (Medio):** Un abuso automatizado podría degradar el rendimiento del servidor.

Desarrollo Técnico: Procedimiento, Payload y Evidencias

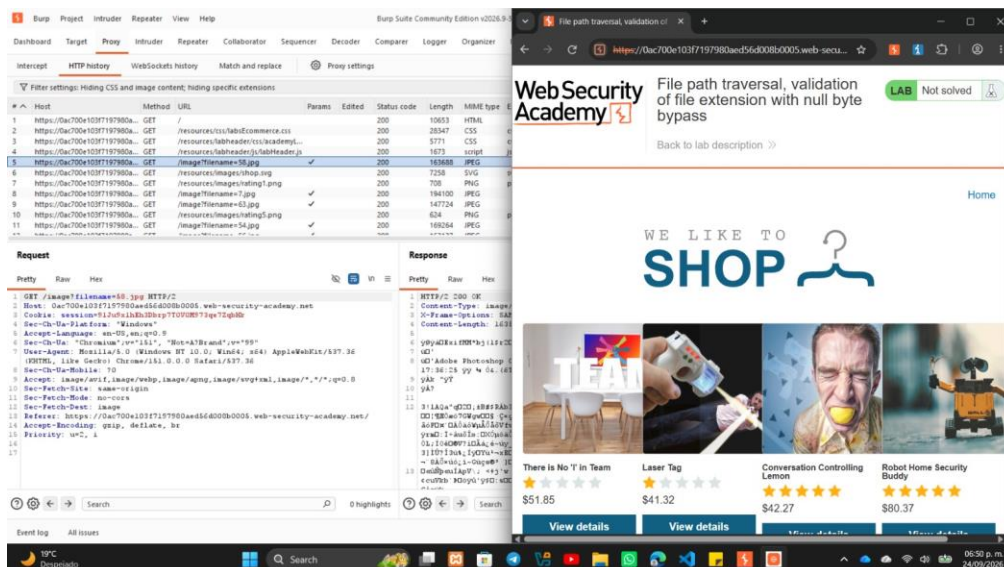
Para explotar este fallo, utilicé Burp Suite siguiendo estos pasos:

Interceptación del tráfico

Entré a la página del laboratorio, copié la URL de la tienda y la pegué en mi navegador configurado con el proxy



Mientras navegaba por los productos, busqué en el historial HTTP de Burp Suite la petición GET encargada de cargar las imágenes mediante el parámetro filename



Mandé una de estas solicitudes válidas (filename=7.jpg) al *Repeater* y comprobé que cargaba correctamente con un 200 OK.

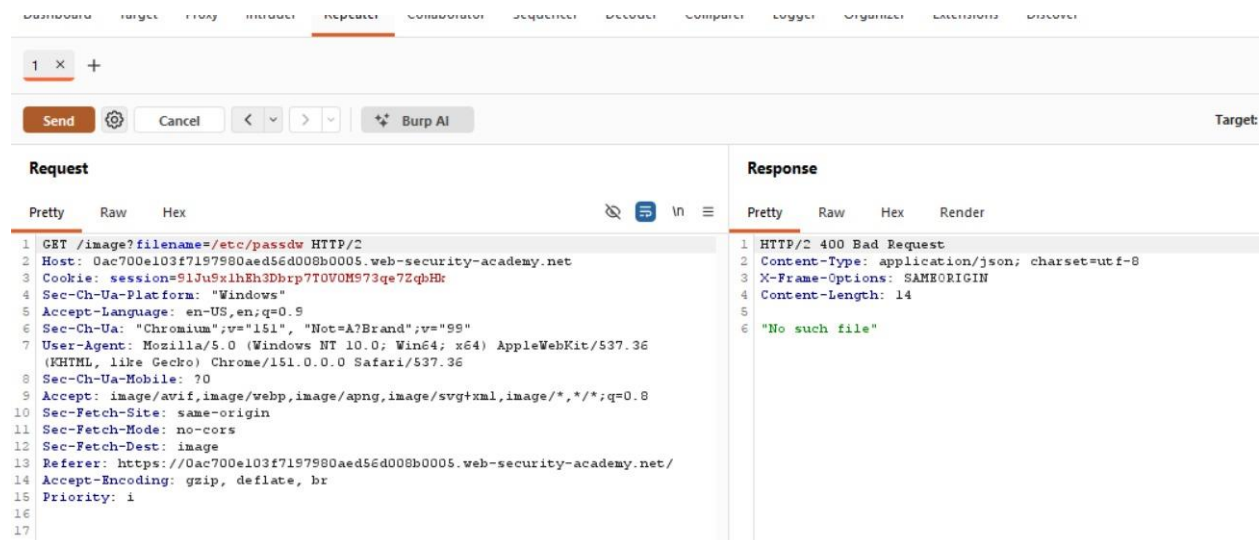
The screenshot shows the Burp Suite interface with the 'HTTP history' tab selected. A table lists several HTTP requests. The 8th request, with URL `https://0ac700e103f7197980a.../image?filename=7.jpg`, is highlighted. A right-click context menu is open over this request, showing options like 'Send to Repeater' (Ctrl+R), 'Send to Sequencer', 'Send to Comparer', 'Send to Decoder', 'Send to Organizer' (Ctrl+O), 'Record an issue', 'Open response in browser', 'Request in browser', 'Engagement tools', 'Cut' (Ctrl-X), 'Copy' (Ctrl-C), 'Copy prettified' (Ctrl+Alt+C), 'Save selected text to file', 'Paste' (Ctrl+V), 'URL-encode as you type', 'Convert selection', 'Copy URL', 'Copy as curl command (bash)', 'Copy links in response', 'Save item', and 'Add to site map'. The 'Inspector' panel on the right shows the request details, including headers and body. The bottom status bar indicates 'Memory: 175.7MB of 7.83GB'.

Pruebas de seguridad y bloqueos

Para probar las defensas, intenté pedir directamente el archivo `/etc/passwd`. El servidor inmediatamente me regresó un error 400 Bad Request indicando "No such file".

The screenshot shows the Burp Suite 'Repeater' tab. A single request is shown in the 'Request' pane, with the URL `https://0ac700e103f7197980a.../etc/passwd`. The 'Response' pane shows the server's reply: `HTTP/2 400 Bad Request`, with headers `Content-Type: application/json; charset=utf-8`, `X-Frame-Options: SAMEORIGIN`, and `Content-Length: 14`. The body of the response is `"No such file"`. The bottom status bar shows the target URL as `https://0ac700e103f7197980a...`.

Luego, intenté un salto de directorio normal (../../../../etc/passwd) sin la extensión correcta, y también falló con el mismo error, demostrando que el servidor valida que el texto termine con la extensión de imagen válida.



Armado del payload con Byte Nulo

Como sabía que el servidor validaba el final de la cadena, pero el sistema de archivos corta la lectura al encontrar un byte nulo, armé el siguiente payload:

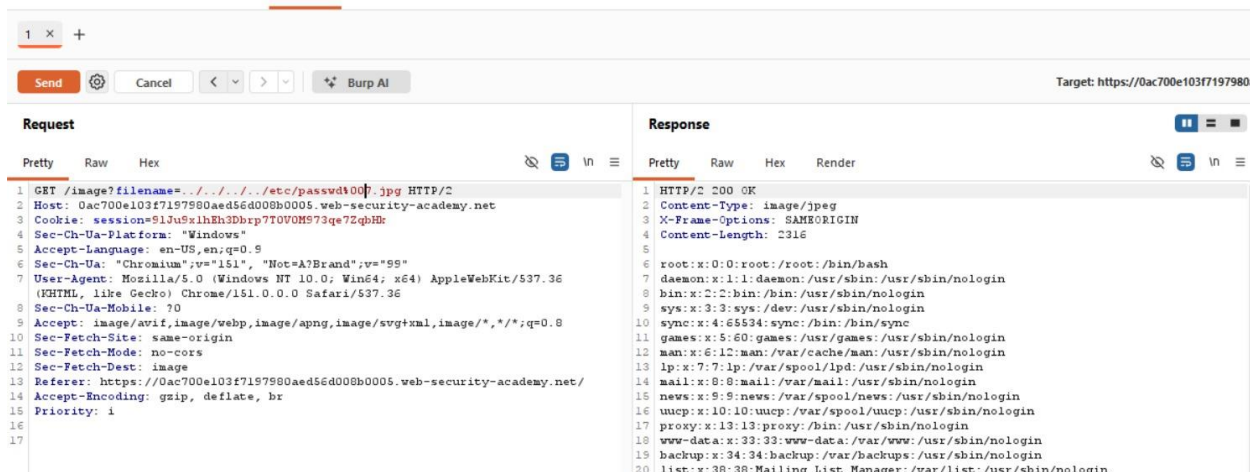
Payload utilizado: “ ../../../../etc/passwd%00.jpg “

La lógica de este ataque funciona en dos pasos:

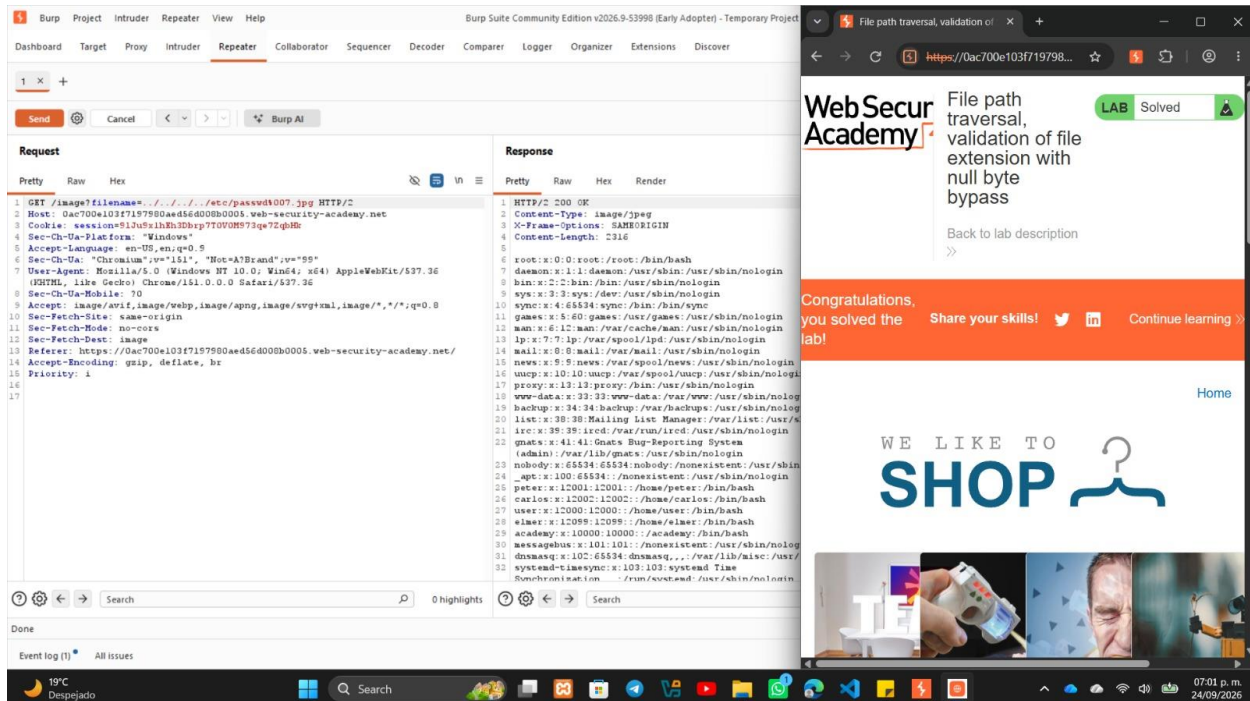
- **El engaño (Validación web):** Al enviar la ruta ../../../../etc/passwd%00.jpg, el filtro de la página revisa el final del texto. Como ve que termina en .jpg, lo aprueba creyendo que es una imagen inofensiva.
- **La explotación (Sistema Operativo):** La aplicación le pasa esta cadena al sistema de archivos para abrirla. El sistema empieza a leer la ruta saltando directorios hasta etc/passwd, y de repente encuentra el %00 (byte nulo). Al verlo, el sistema asume que ahí terminó el nombre del archivo y **descarta todo lo que sigue** (ignorando el .jpg). Al final, abre únicamente ../../../../etc/passwd.

Exfiltración y confirmación

Injecté mi payload en el parámetro filename del *Repeater* y lo envié. Gracias al truncamiento provocado por el byte nulo, el sistema ignoró la extensión y ejecutó mis saltos de directorio de forma exitosa, respondiendo con un HTTP/2 200 OK y exponiendo todo el archivo de usuarios del sistema.



Al revisar la página web en mi navegador, apareció el banner naranja de “Congratulations, ¡you solved the lab!”



actualizando oficialmente el estatus de la práctica a "Lab Solved"



Estrategias de Mitigación Recomendadas

Para tapar esta vulnerabilidad estructural, se deben aplicar las siguientes defensas:

- **Rechazar Bytes Nulos:** La aplicación debe sanitizar y rechazar automáticamente cualquier solicitud que contenga un byte nulo (%00, \0), ya que no es un carácter válido para el nombre de un archivo legítimo.
- **Validación estricta de la extensión (Listas Blancas):** No basta con revisar si la cadena "termina en .jpg". Se debe validar que *toda* la cadena sea segura, permitiendo solo letras, números y una única extensión al final usando expresiones regulares estrictas (ej. `^[a-zA-Z0-9]+\.(jpg$)`).
- **Canonicalización de rutas:** El servidor debe usar funciones nativas para convertir la ruta ingresada a su forma absoluta y final. *Después* de obtener esa ruta final real (ya sin bytes nulos ni saltos `../`), debe comprobar que no se haya salido del directorio base (`/var/www/images/`).
- **Referencias indirectas:** Evitar que el usuario pida archivos por su nombre físico en el disco. Es mejor usar identificadores internos de base de datos.

Conclusión y Aprendizaje Técnico

Este laboratorio me demostró un principio crítico en la ciberseguridad: la diferencia entre cómo un filtro de aplicación lee los datos y cómo los procesa el sistema operativo a bajo nivel.

Crear filtros que solo se fijan en un fragmento de la información (como revisar únicamente cómo termina el texto) es un error arquitectónico grave. Burlar esta seguridad fue posible porque engañamos al filtro pasándole la extensión que quería ver, pero usando el Byte Nulo (%00) forzamos al sistema de archivos a truncar la

cadena y descartar esa misma extensión a la hora de buscar el archivo en el disco duro.

En el desarrollo seguro, no podemos confiar en validaciones superficiales de texto. Las protecciones deben contemplar cómo interactúan las distintas capas tecnológicas (código web, servidor y sistema operativo) y siempre desinfectar la entrada de caracteres de control maliciosos.